An MLIR Backend for a Simulated Edge NPU: Debug Report

Olajide Badejo

August 15, 2026

Abstract

This companion to the main report is a first person postmortem of the problems met while building the compiler and how they were resolved, grouped by theme. It is drawn from the engineering log kept from the first phase. The intent is to record the real friction, the environment mismatches, the ODS surprises, and the one genuine correctness bug, so that the reasoning behind the fixes is not lost.

1 Build system and environment

The build environment did not match the specification's assumptions on almost every axis, and one mismatch was safety critical. The machine runs its Linux toolchain under WSL2 with a deliberate resource cap of twelve gigabytes of memory and eight processors, set after an earlier run without a cap exhausted host memory and destabilized the machine. The specification suggested a larger budget and twenty parallel compile jobs. Following that would have reintroduced the exact condition that had caused the crash, so instead the one time LLVM and MLIR build was configured with six compile jobs and a single link job, which is the guidance in the resource cap file itself. The build completed in about twenty eight minutes and stayed near three gigabytes of resident memory.

A second environment problem cost real time. The project folder path contained spaces, and the LLVM lit and FileCheck test tools do not support spaces in paths; they resolve symlinks, so a space free symlink over the spaced directory did not help either. The fix was to host the repository in the WSL native filesystem, which is also much faster to build in than the mounted Windows filesystem.

Finally, the entry points `registerAllDialects` and `registerAllPasses` now live in their own aggregator libraries in this LLVM release and are not pulled in by the dialect library properties, so the first link of the optimizer driver failed with undefined references until those libraries were listed explicitly.

2 Dialect and ODS surprises

Two failures while defining the ops were of the same family: the TableGen half and the C++ half of an interface are separate includes. The convenience trait `SameOperandsAndResultType` is not in the base op definitions; it lives in the infer type interface definitions, so the first build failed with an undefined TableGen variable until that file was included. After fixing that, the C++ build failed because the same trait mixes in the infer type interface, so the generated op class references a C++ interface whose header and link library also had to be added. The lesson, recorded for the later ops, is that adding a trait in ODS may drag in an interface with a matching header and library, and both should be added at the same time rather than after the next build failure.

A third surprise was in constant folding. The elementwise folders use the common folder helpers, which in this release gained a poison attribute template parameter that defaults to a type only defined if the undefined behavior dialect is linked. The parameter is the third one, so opting out of poison propagation meant passing all three template arguments explicitly rather than the natural one. This avoids taking a dependency on a dialect that these folds do not need.

3 Lowering and numerics

The lowering from the tensor level to the instruction level is a representation change, and the dialect conversion framework is the right tool for it, but the framework's materialization rules took care to get right. A type converter maps every tensor to a scratchpad buffer, and target and source materializations insert a load or a store at the boundary. The subtlety is keeping DRAM values, the function arguments and the constants, as tensors while the intermediates become buffers, so that DMA appears only where the two memories meet. A constant becomes a DRAM constant followed by a load, which makes the boundary explicit.

The one genuine correctness bug appeared only under a tight scratchpad budget, and the benchmark suite caught it. At a small budget the allocator spills a buffer by storing it to DRAM after it is produced and reloading it before its next use. The maximum error against onnxruntime jumped from the fp32 rounding level to about eight hundredths. The cause was in the instruction encoder, which assigned DRAM offsets only to inputs, constants, and returned values. A spill store's result is a DRAM temporary that is none of those, so with no offset assigned it defaulted to offset zero and overwrote the model input, and the reload read the input back instead of the spilled buffer. The scratchpad allocation test had checked only that spilling inserted the right number of DMA instructions, never the numerics, so it did not catch this. The fix gives every spill temporary its own DRAM region, and an end to end test at a tight budget now locks the numerics in.

4 Tooling

The Python environment was newer than the specification assumed, running Python 3.14. This was a real risk for the frontend, since a compiler that cannot import its own dependencies is stuck at the first phase. The risk was retired early by confirming that numpy, nanobind, onnx, onnxruntime, and a CPU build of torch all publish wheels for this interpreter, so the frontend proceeded on the existing environment rather than a downgraded one.

On the C++ side, LLVM builds with exceptions and run time type information disabled. The first version of the binary decoder used exceptions to signal a malformed file, which did not compile. Rather than fight the build flags per target, the decoder was refactored to return an optional, which is a cleaner interface anyway and bounds any corrupt length field so a bad file cannot request a huge allocation.

A smaller but recurring friction was invoking the Linux toolchain from the Windows side. Multi line commands passed through the shell bridge had their quoting mangled, so the reliable pattern became writing a short script to a scratch location and running that script, rather than passing complex command lines inline.